

CS 3510 Algorithms: NP-Complete: Graph Coloring

Aaron Hillegass
Georgia Tech

P is the set of decision problems solvable in polynomial time.

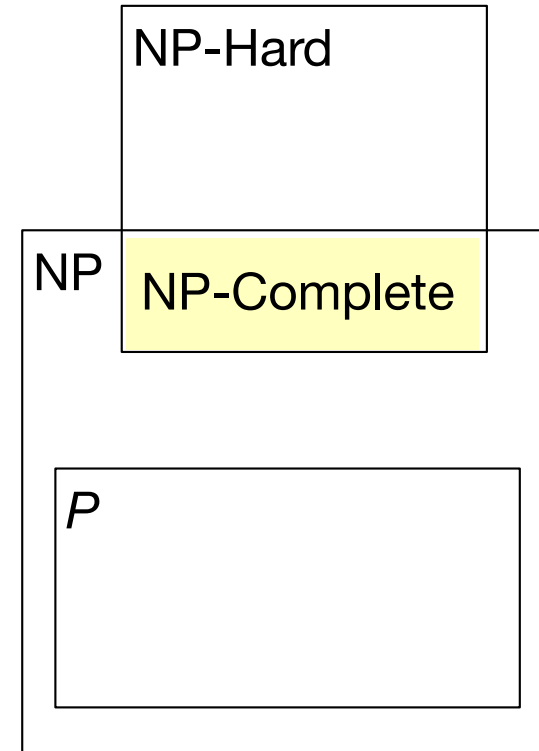
NP is the set of decision problems verifiable in polynomial time.

When we reduce a problem H to a problem J , we are saying " J is at least as hard as H "

An easy problem like RELPRIME is reducible to a hard problem like SAT

A problem H is in NP-Hard? *Any* problem in NP can be reduced to H .

The intersection of NP and NP-Hard is NP-Complete.



Cook-Levin Theorem gave us one problem in NP-Complete: SAT

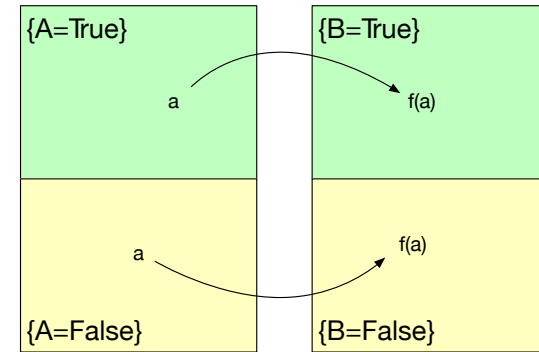
Every decision problem in NP can be reduced to SAT.

Proving a problem H is in NP-Complete is two steps:

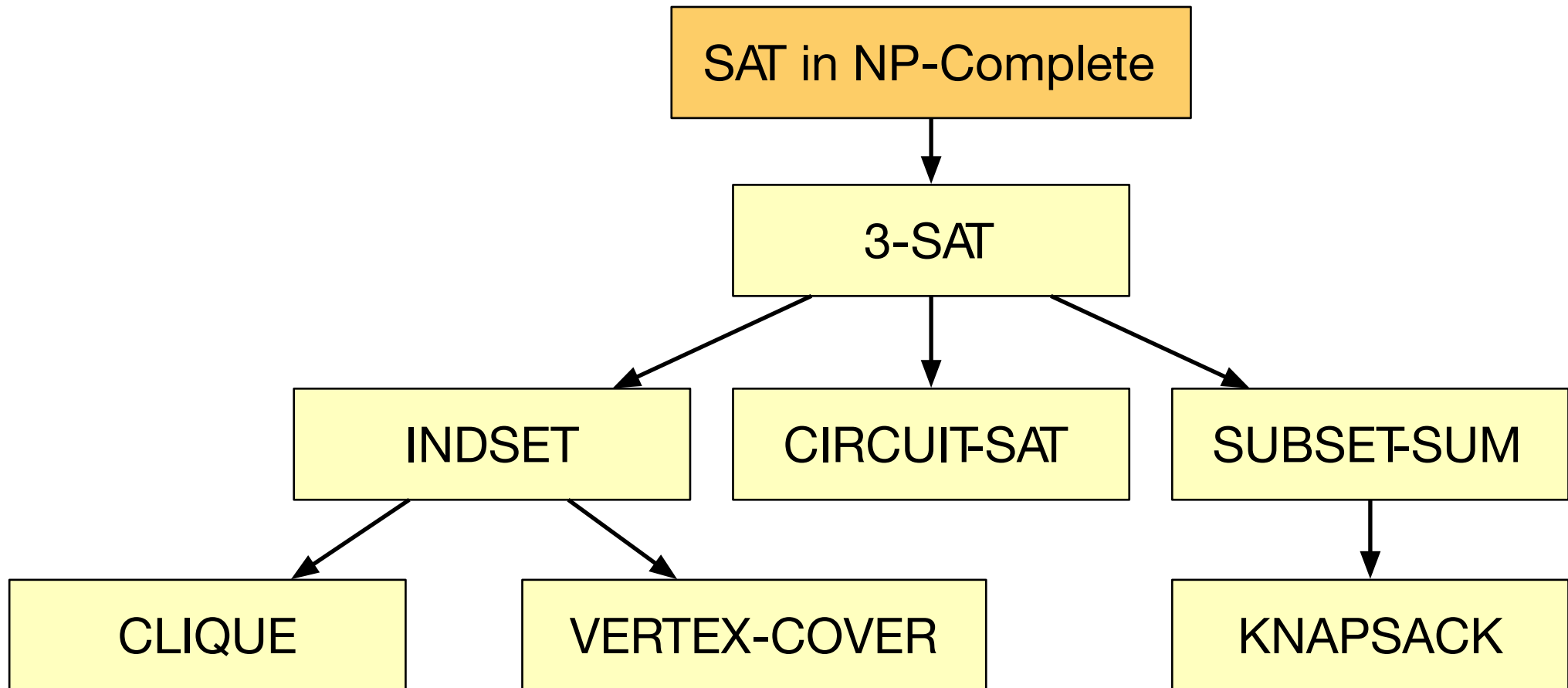
- Show that the problem is in NP
- Show SAT (maybe indirectly) reduces to the problem

That is, given an expression Φ_S in SAT, you can convert it into an instance Φ_H in H in polynomial time

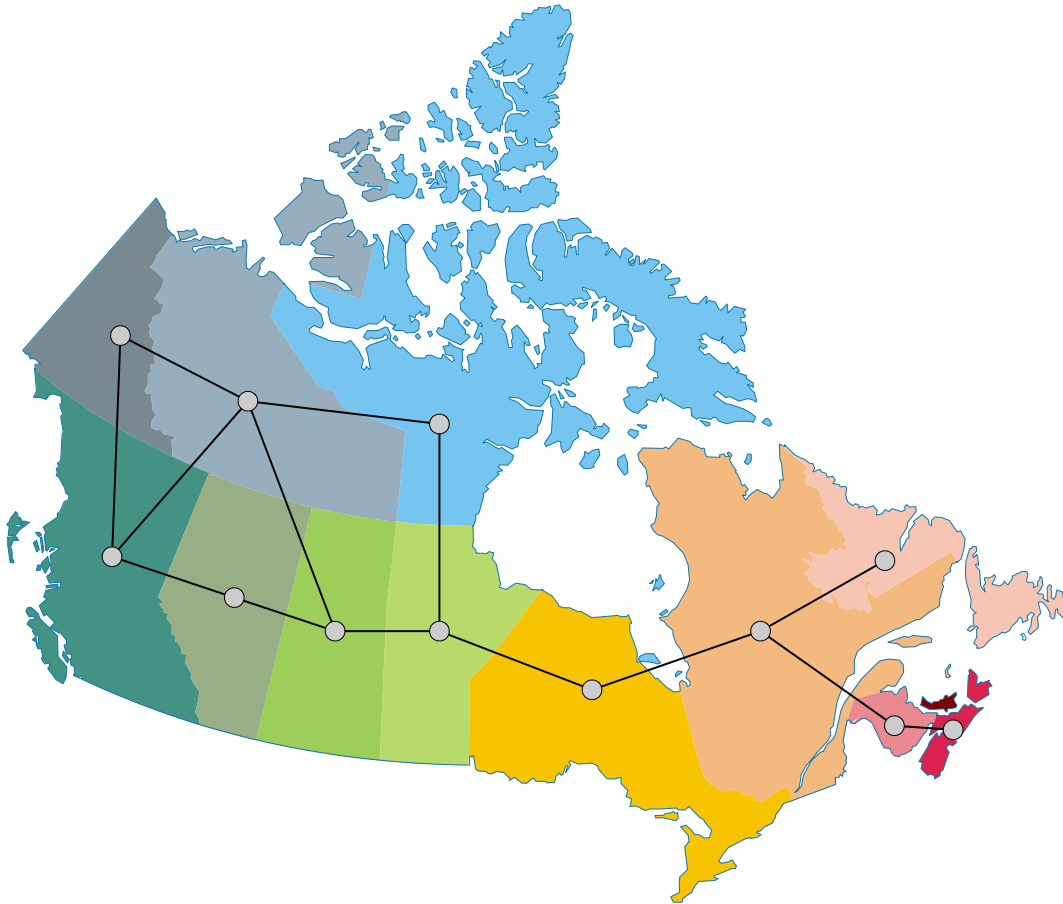
And Φ_H is true if and only if Φ_S is true.



So far:



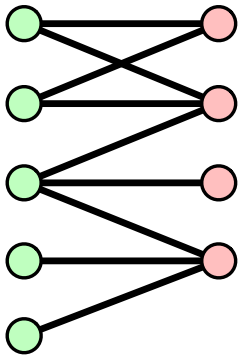
Graph Coloring



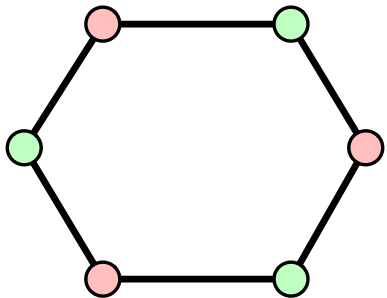
Given a graph G and a number of colors k :

- Each vertex is assigned a color from $1..k$.
- No two adjacent vertices have the same color.

Two Colorable?



Any class of graphs that you can be sure are 2-colorable?



So what is the runtime to determine 2-colorable?

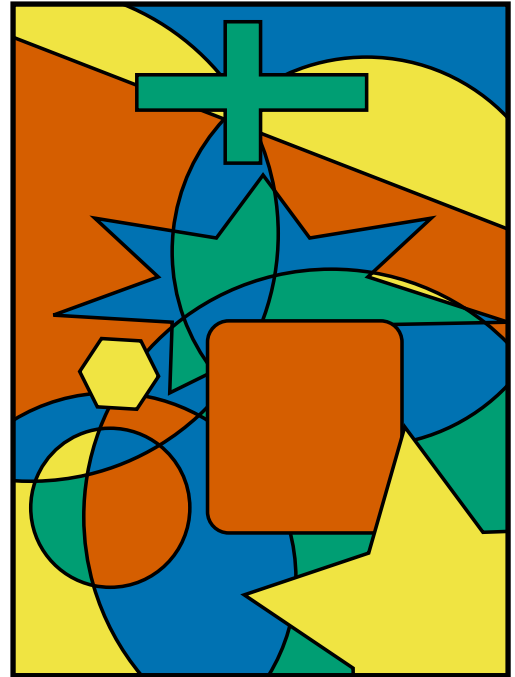
Four Colorable?

7/24

If a graph can be laid out on a plane with no line crossing, it is said to be *planar*.

Four-color Theorem: Every planar graph is 4-colorable.

(So every map is 4-colorable.)

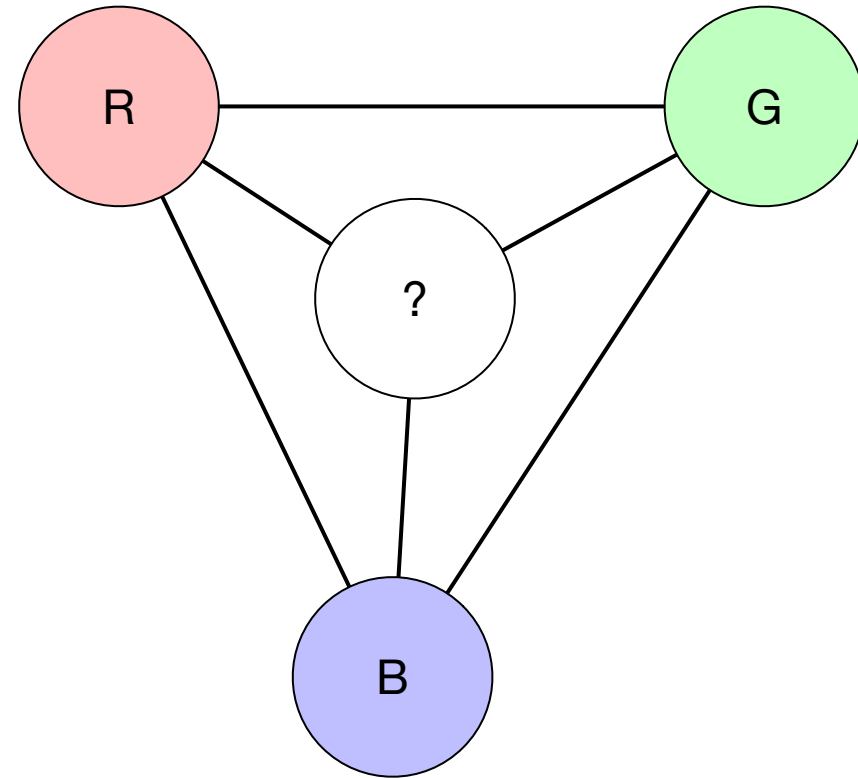


Three Colors?

The problem: Given G , is it 3-colorable?

Is it in NP?

It is NP-Complete. What can we reduce to 3-coloring?



Reducing 3SAT to 3-Color

Given an expression Φ in SAT, we can convert it into a graph G such that G is 3-colorable if and only if Φ is satisfiable?

Let's use Tangerine, Fuschia, and Silver as colors.

We will have nodes in the graph that represent each of the literals:

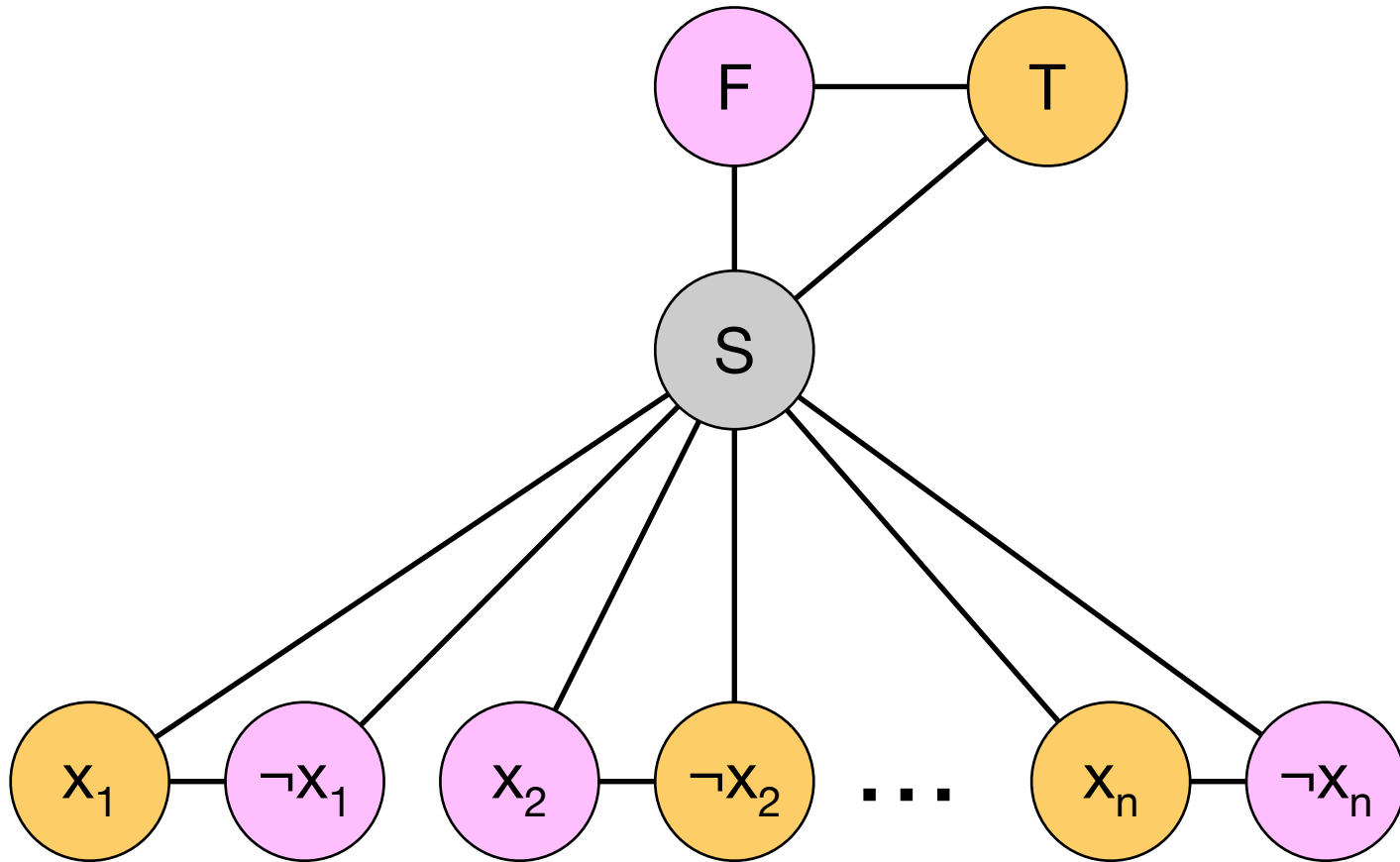
$$x_1, \neg x_1, x_2, \neg x_2, \dots, x_n, \neg x_n$$

How can we guarantee that for an x_i , either

- $x_i = T$ and $\neg x_i = F$ or
- $x_i = F$ and $\neg x_i = T$

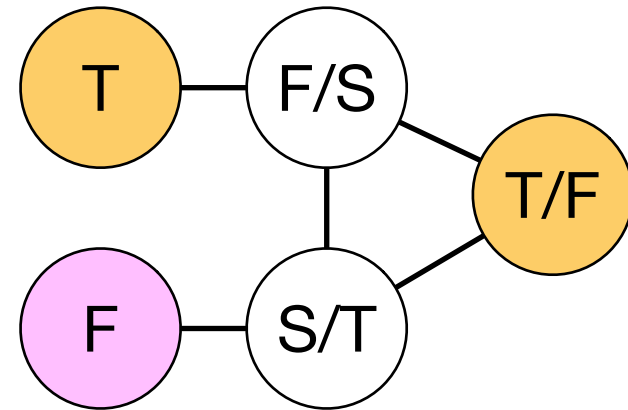
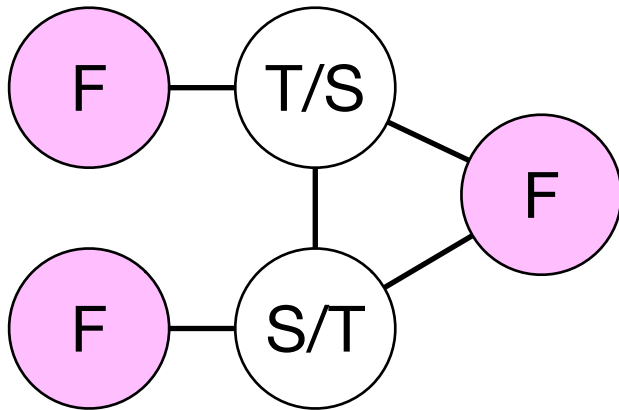
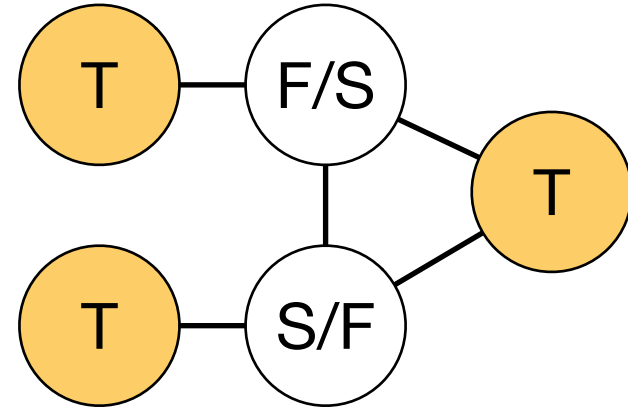
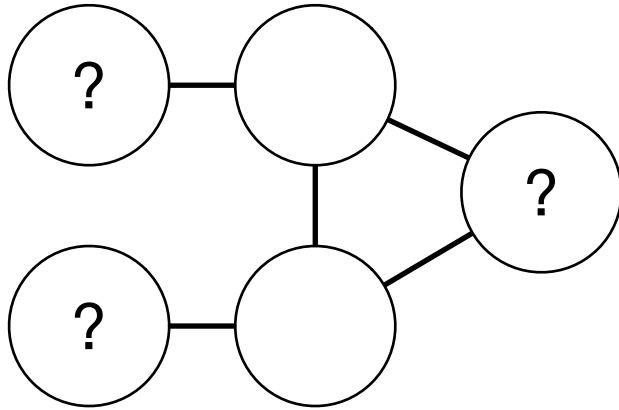
Boolean Restrictions in 3-Colors

10/24

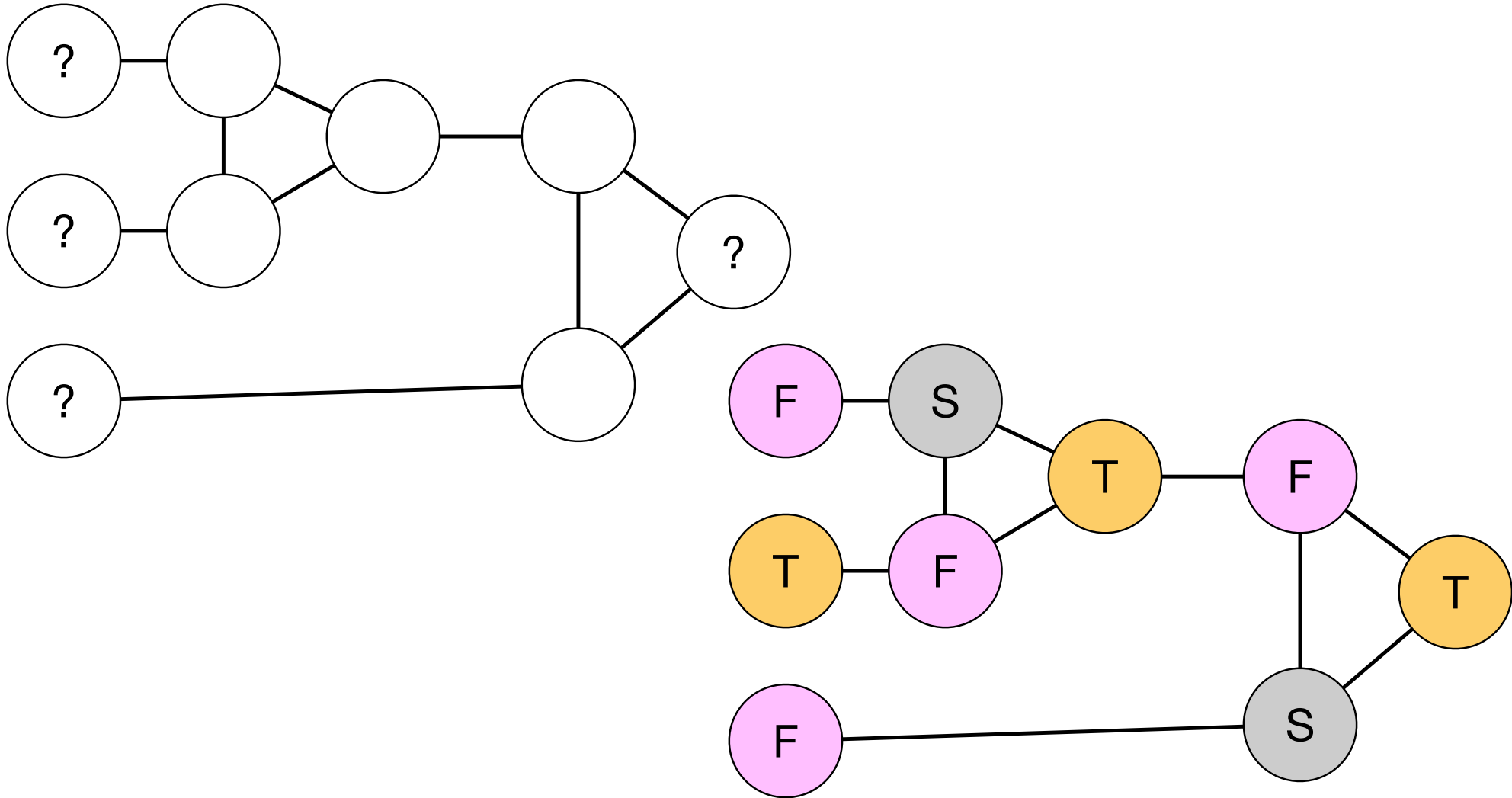


OR-gate in 3-Colors Gadget

11/24



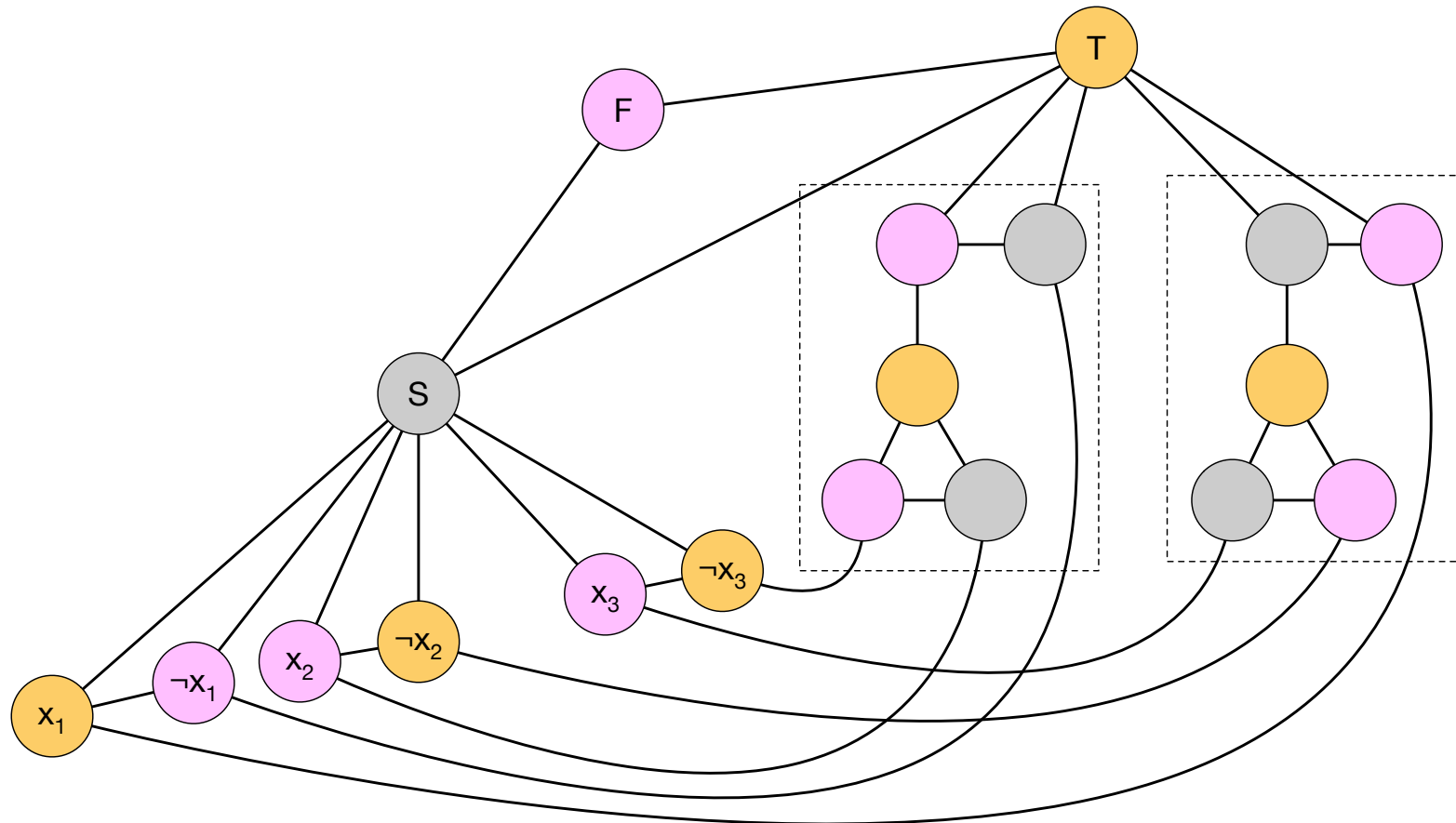
3-Clause Gadget



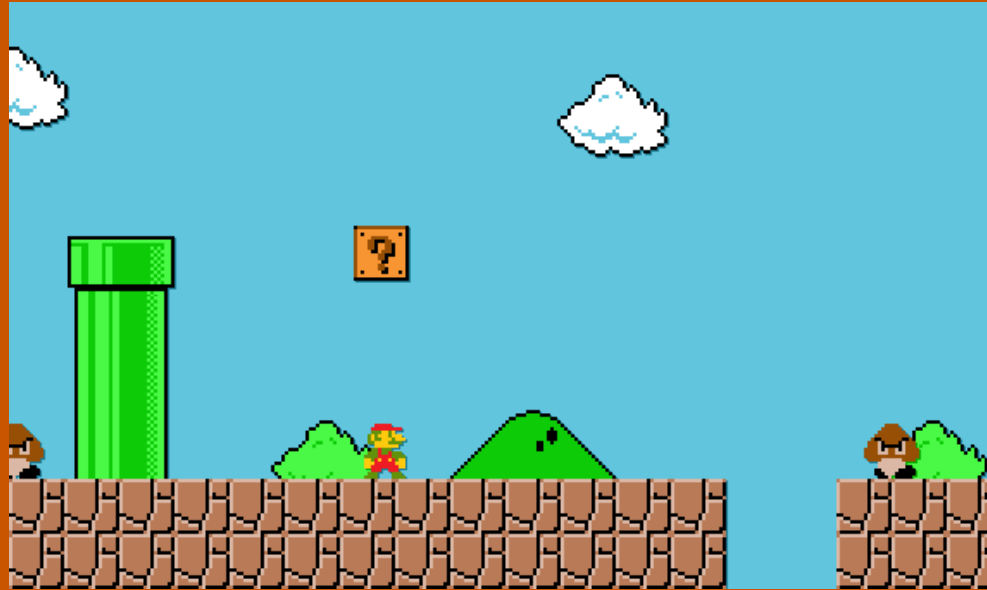
How do we require each clause to be true?

The Whole Graph

$$\Phi = (\neg x_1 \vee x_2 \vee \neg x_3) \wedge (x_1 \vee \neg x_2 \vee x_3): x_1 = T, x_2 = F, x_3 = F$$



Is Super Mario Brothers NP-Complete?



Erik Demaine is the king of showing found objects are NP-Hard.

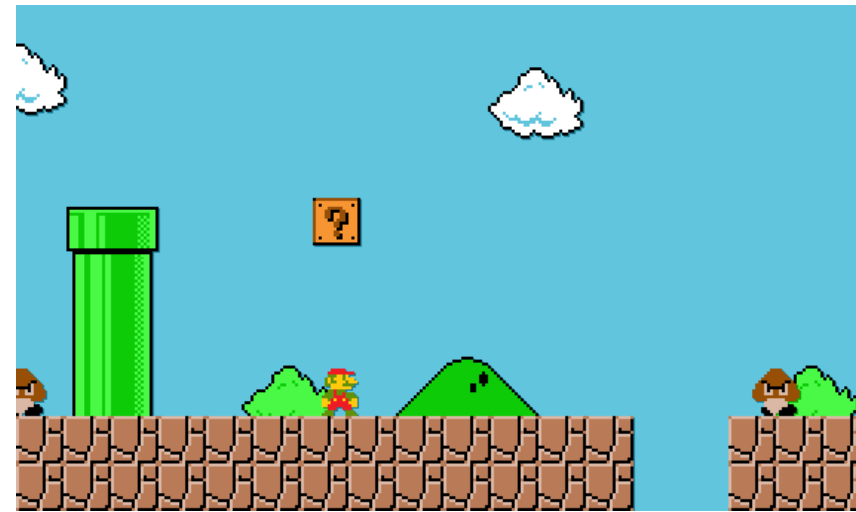
Reducing 3SAT to MARIO

For every Φ instance in 3SAT, can we make a MARIO level that can be completed if and only if Φ is satisfiable?

We can!

So MARIO is NP-Hard, but probably not in NP.

<https://youtu.be/oS8m9fSk-Wk?si=Av5Ndmel0en5n4zl>

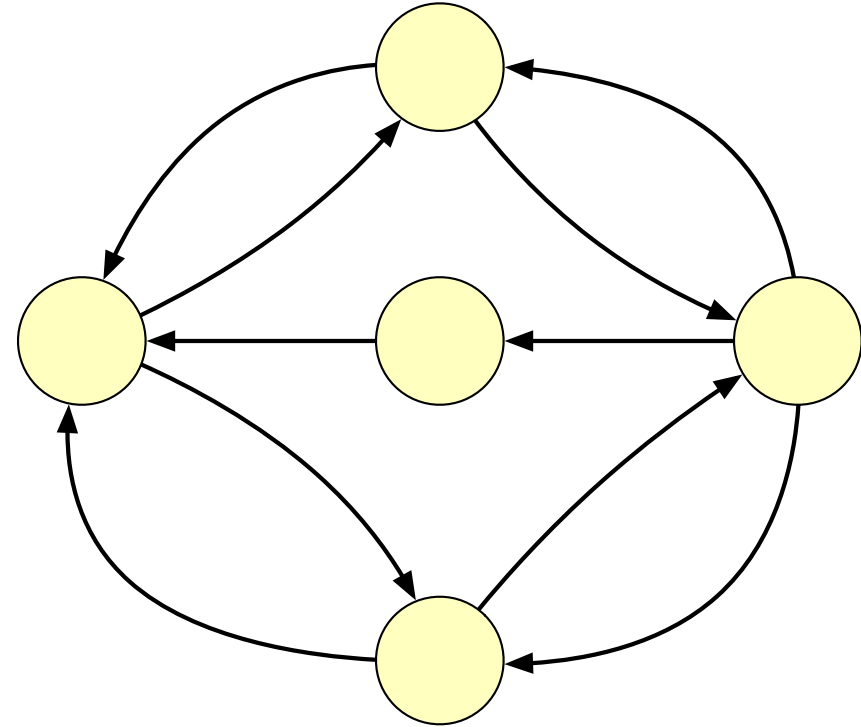


Hamiltonian Cycle Problem

17/24

Given a directed graph G is there a path that starts and ends at the same vertex and touches every other vertex exactly once?

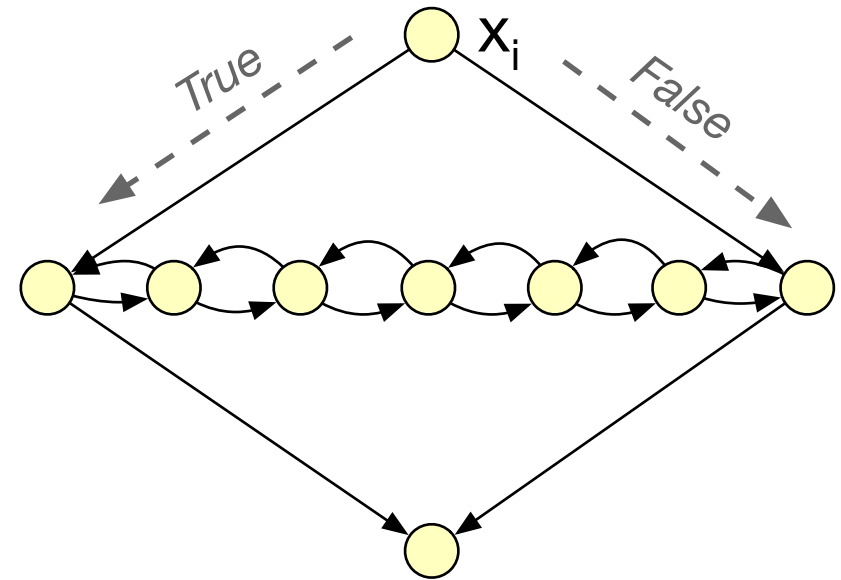
Is it in NP?



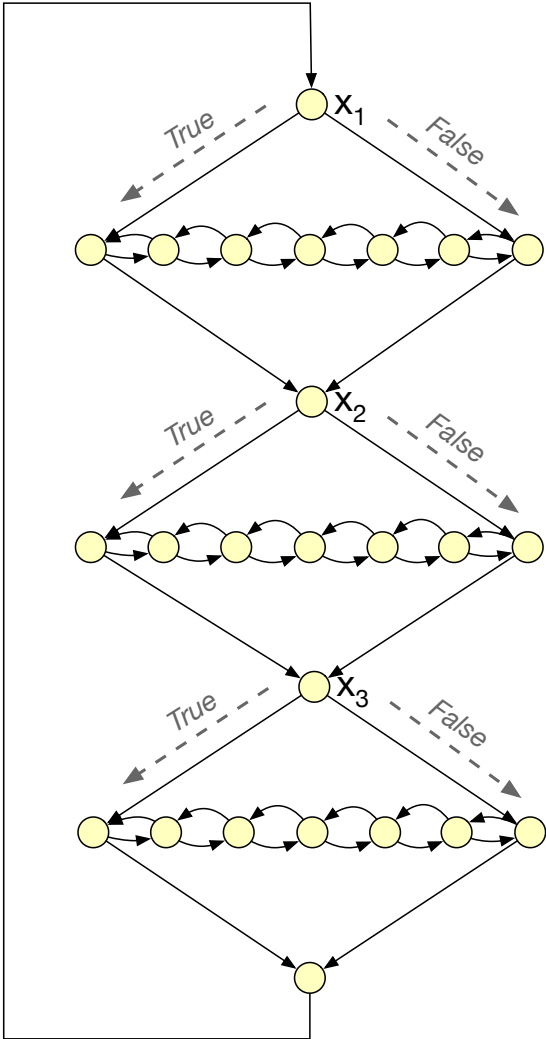
Reducing 3-SAT to Hamilton Cycle

Given a 3-SAT expression Φ can we make a graph G that will have a Hamiltonian cycle if and only if Φ is satisfiable?

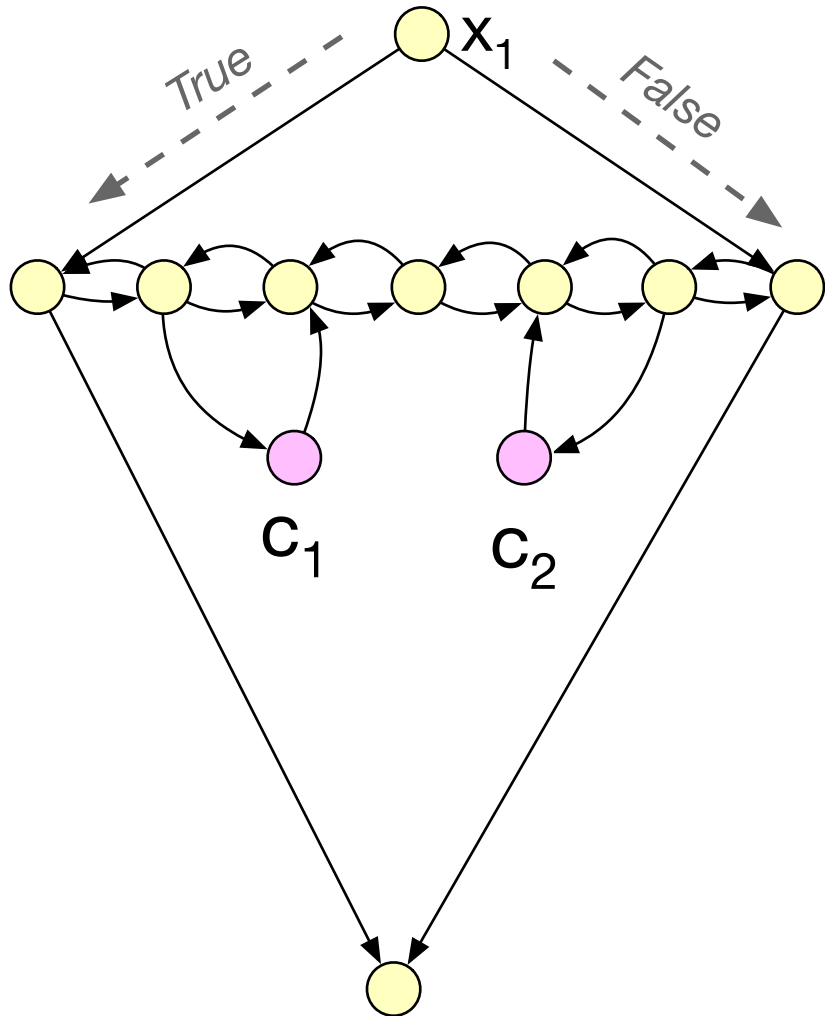
Here is the gadget we will use for each variable.



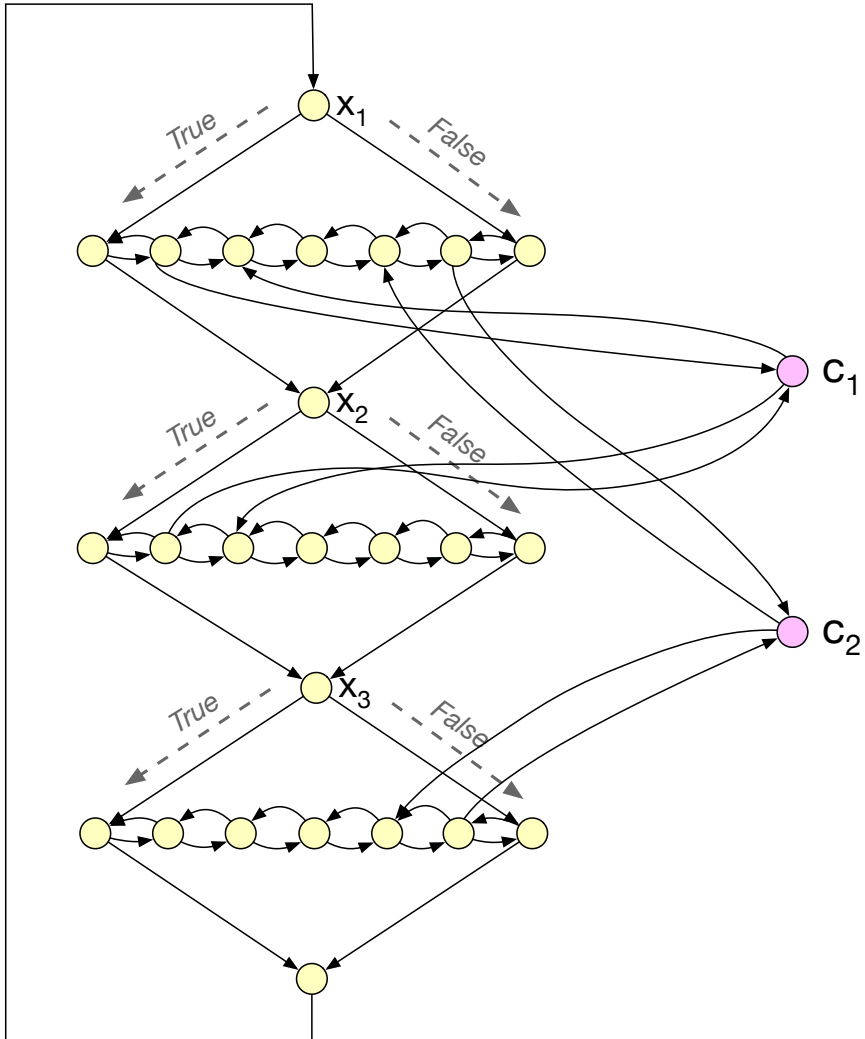
Assigning all the variables



Clauses become vertices



More than one var per clause



Travelling Salesperson Problem

Given a directed graph G with weighted edges and a number k , is there a circuit that visits every vertex exactly once and has a total cost less than or equal to k ?

Hamiltonian Cycle reduces to TSP

Given a graph G , how can we use TSP to determine if G has a Hamiltonian cycle?

We can use the following reduction:

1. Create a complete graph G' with the same vertices as G .
2. Assign a weight of 1 to each edge in G , zero to each edge not in G .
3. Set k be zero.

If G has a Hamiltonian cycle, then G' has a circuit that visits every vertex exactly once and has a total cost of 0.

Questions?

Slides by Aaron Hillegass

Based on lectures by Abraham Ladha